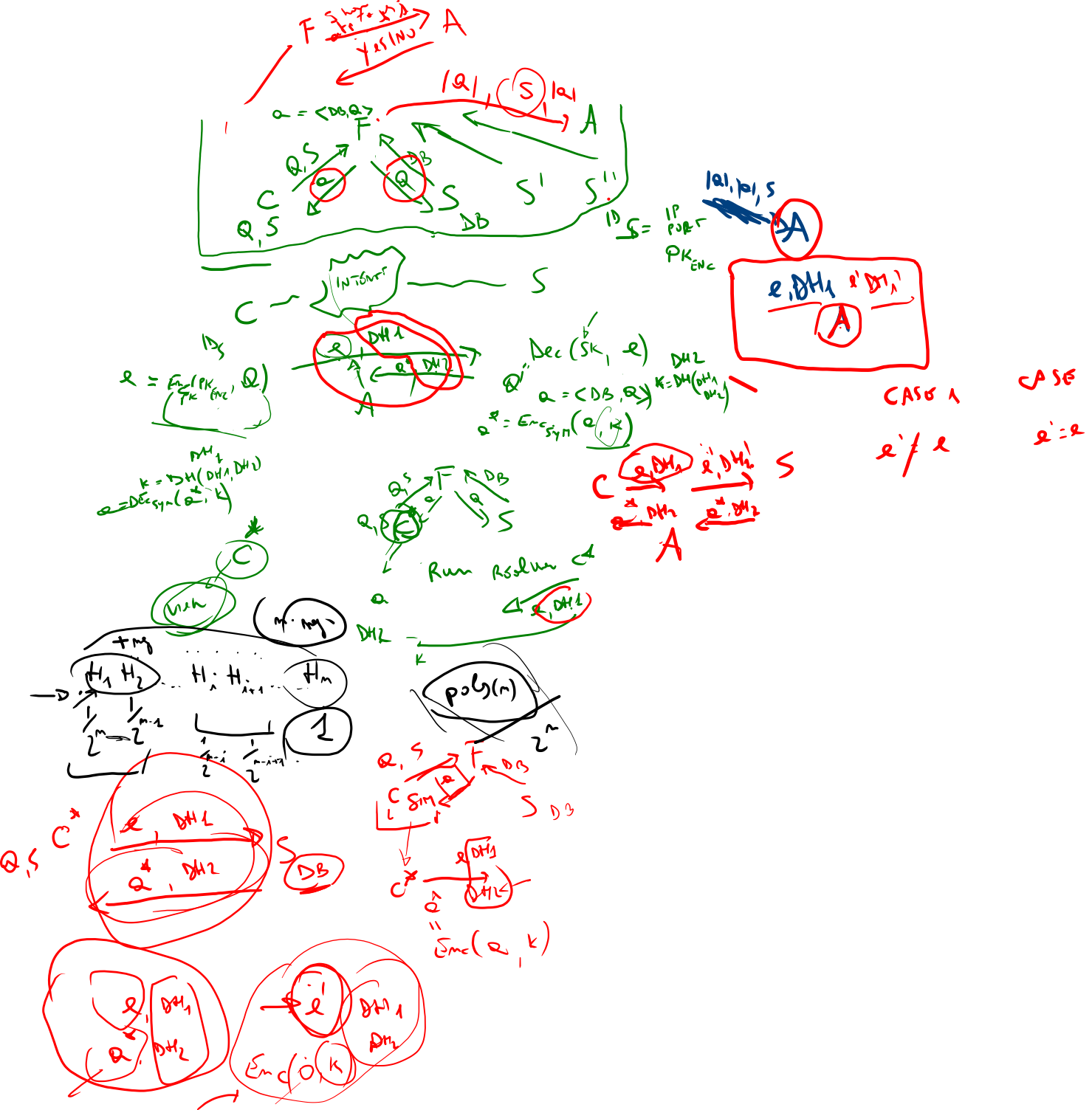
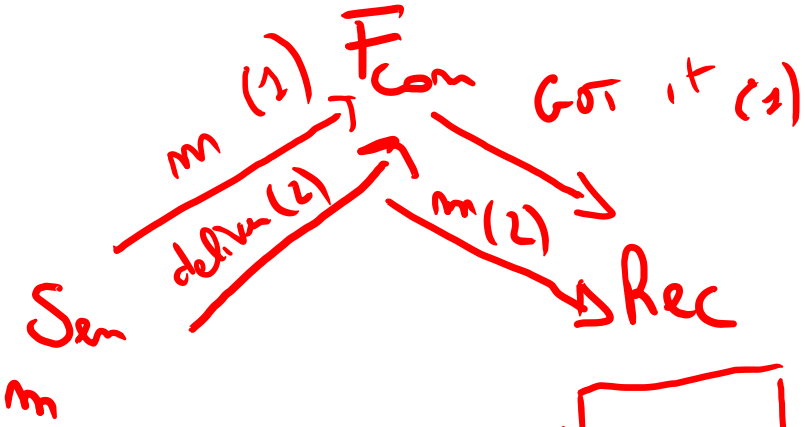
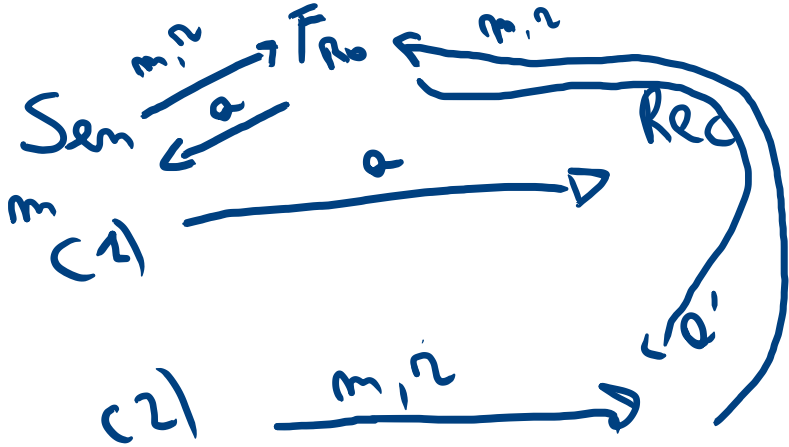

F
A
C
S
S'
S''
DB
INTERNET
DH1
DH2
A
CASO 1
CASO
e' ≠ e
e' = e
Run Resolver
poly(n)

PHASE 1

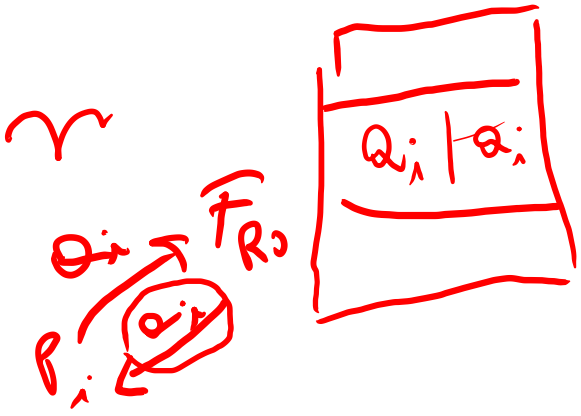




if $a' == a$

return m

PHASE 2



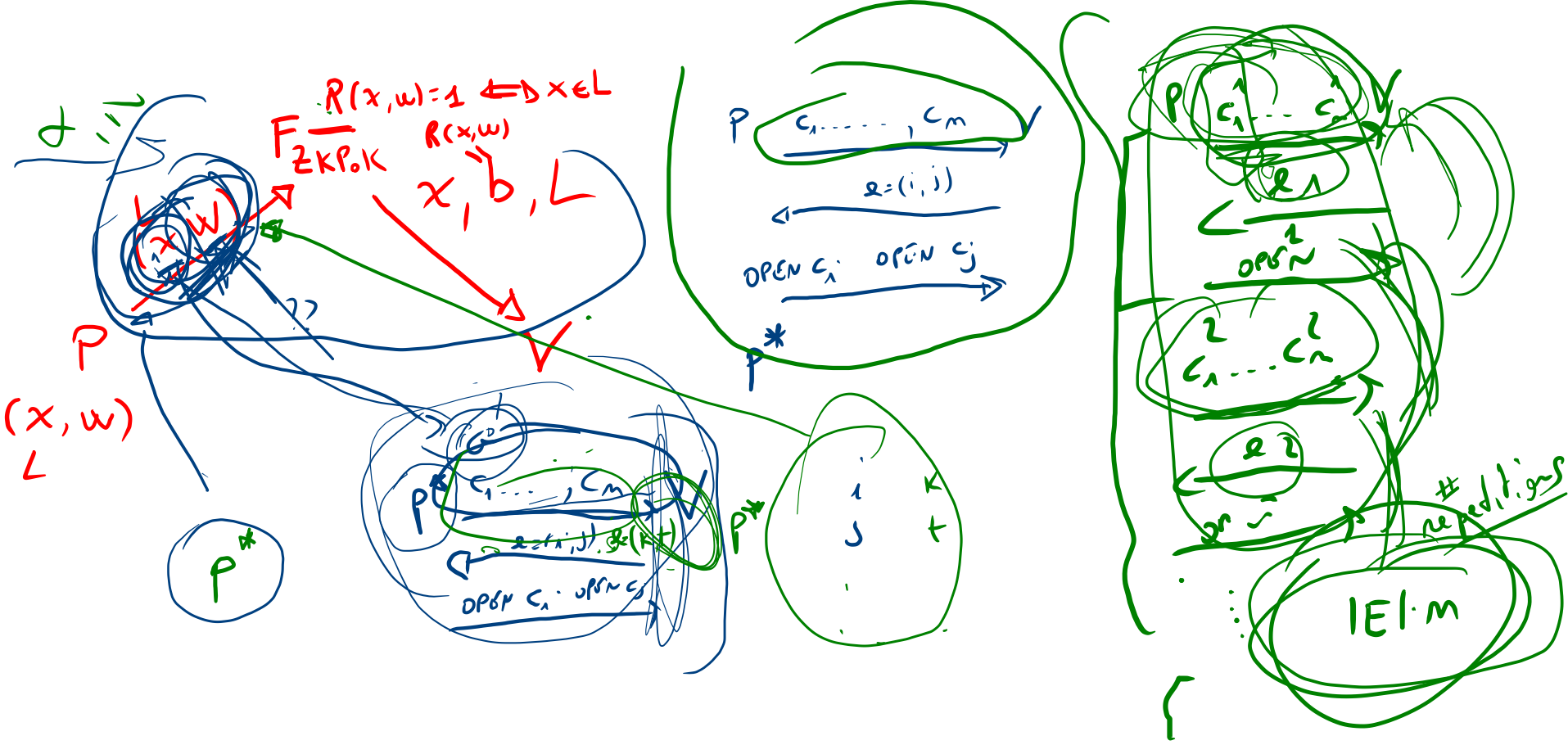
$\langle$ S is fully malicious
R is fully malicious

$\lambda R \xrightarrow{\text{random}} R$

m

Com(m): $r \leftarrow \{0,1\}$

return SHA256(m|r)

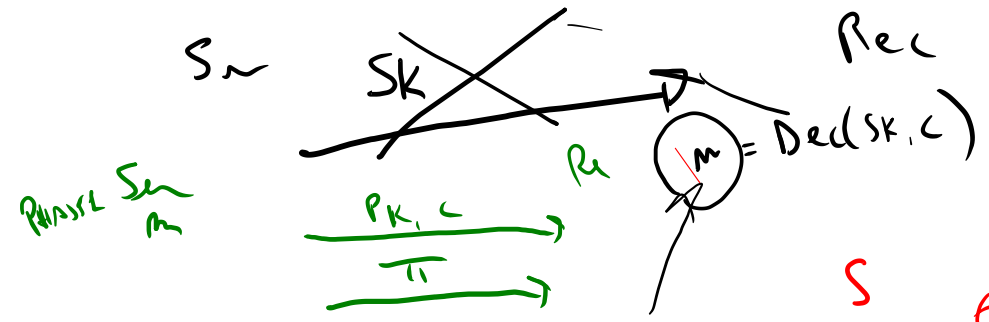

R(x,w)=1 ⟺ x∈L
F_ZKPoK
R(x,w)
x, b, L
P
(x,w)
L
P*
P
c_1, ..., c_m
V
e=(i,j)
OPEN c_i · OPEN c_j
P*
P
c_1, ..., c_m
V
e=(i,j), e∈(k,t)
OPEN c_i · OPEN c_j
P*
i
k
j
t
P
c_1^1 ... c_n^1
e_1
OPEN^1
c_1^2 ... c_n^2
e_2
OPEN^2
repetitions
|E|·m

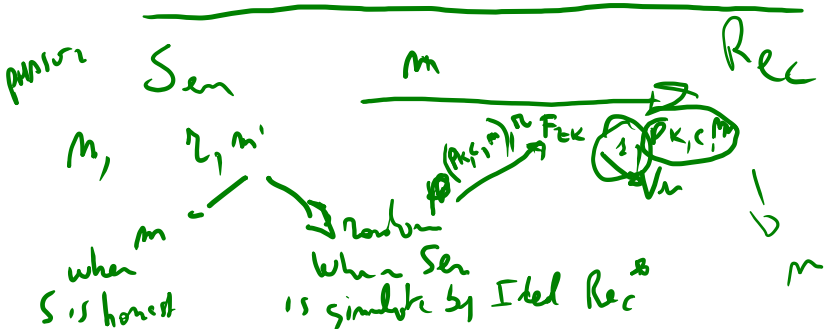
GOAL (Sen, Rec)

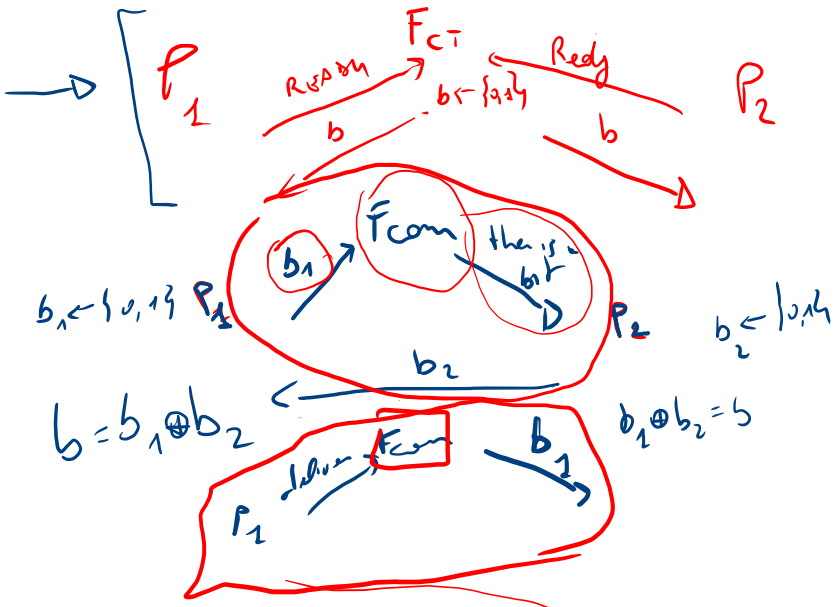
through – PK CPA encryption (Gen, Enc, Dec) with self verifiable keys

– ZKPoK functionality

Real world protocol







P_1^* real-world

P_1^* ideal world

commitment realize the F_{com} functionality

P_1^* run P_1^* behave as P_2

P_1^* ready "there is a bit"

P_1^* gets b_1 extracting

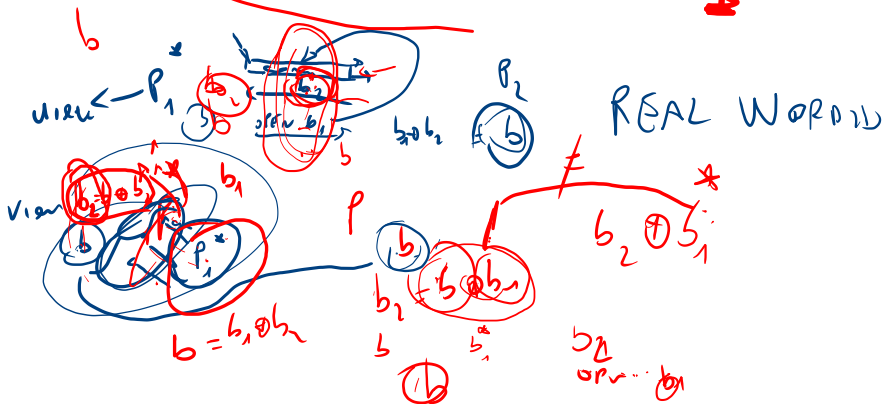
(or alternatively we can see P_1^* sitting in the place of F_{com})

pick $b_2 = b \oplus b_1$ send it to P_1^*

get opening and output what P_1^* outputs

P_2^*

P_2^* (run as such so that P_2^* knows there is a commitment bit)

get b_2, get b from functionality, compute $b_1 = b \oplus b_2$

run the opening delivery b_1